Routing Overhead and Escalation Misconfiguration in Hybrid LLM Routers: An Empirical Study with a Data-Driven Fix

First Author
Affiliation / Institution
first@hydraroute.example

Second Author
Affiliation / Institution
second@hydraroute.example

Third Author
Affiliation / Institution
third@hydraroute.example

Abstract

Hybrid LLM routers that combine a deterministic pre-LLM solver tier with cascaded model routing are widely assumed to be cheaper than single-model serving. We show, through a reproducible 67-task benchmark, that this assumption can be false. Using HydraRoute, a four-tier fallback router (deterministic Tier 0, optional local LLM, small API model, large API model), we first measure a *naive* category-to-size escalation policy and find it *over-escalates*: on actual Gemma-4 OpenRouter pricing the routed system costs **+51.5%** more tokens than an always-large baseline, because 29 of 45 paid tasks are needlessly sent to the large tier. This is a concrete instance of a known finding from RouterBench—that a router can lose to a single fixed model. We then apply a *data-driven fix*: a deterministic Tier 0 offload (solving 22/67 tasks at zero token cost) plus escalation restricted to only the categories that genuinely need a large model (sentiment classification). The fixed router serves 65/67 tasks correctly and, at frontier-model pricing, costs **92% less** than always-large. A vLLM serving benchmark on an RTX PRO 6000 Blackwell GPU confirms the small tier is $\sim$2$\times$ faster, so selective routing is latency-feasible. The contribution is empirical: a negative result (naive routing can cost more), a remedy (deterministic offload + ablated escalation), and a positioning of Tier 0 offload as a more robust alternative to learned routing under small price gaps. We release the system, benchmark, and ablation harness.

1 Introduction

Large language model (LLM) routing has become a central technique for reducing the cost of serving LLMs. Prior systems such as FrugalGPT (Chen et al., 2023), RouteLLM (Ong et al., 2025), MixLLM (Jiang et al., 2024a), and SATER (Shen et al., 2025) share a common assumption: *every incoming query touches at least one LLM* (a small model first, with escalation to a large model on uncertainty). This assumption leaves a large category of queries, those that are fully solvable by deterministic code, paying the full tax of an LLM call, latency, and token cost.

HydraRoute challenges that assumption. It inserts a *deterministic pre-LLM tier* before any LLM is consulted, so that queries matching a known solvable pattern are answered locally and for free. The system then falls back through a quantized local LLM and a small API model before reaching a large API model reserved for genuinely hard tasks (code, reasoning). A key reliability layer is the *SymPy-LLM Symbiosis*: for algebraic word problems, the LLM acts purely as a natural-language-to-equation translator, emitting one SymPy equation string, and the exact arithmetic is delegated to the symbolic solver. This differs from program-aided approaches such as PAL (Gao et al., 2023) and PoT (Chen et al., 2022), which generate general-purpose Python, and from Logic-LM (Pan et al., 2023), which targets logic solvers. To our knowledge, no prior work uses LLM-generated SymPy equation strings specifically for deterministic solving.

This paper makes three contributions. (1) A reproducible *negative* empirical result: on actual Gemma-4 OpenRouter pricing, a naive category-to-size escalation router costs **+51.5%** more than an always-large baseline, because it over-escalates 29 of 45 paid tasks to the large tier—a concrete instance of RouterBench's finding that a router can lose to a single fixed model. (2) A *data-driven fix*: a deterministic Tier 0 offload (22/67 tasks solved at zero token cost) plus escalation restricted to the categories that genuinely need a large model. The fixed router holds 97% accuracy and, at frontier-model pricing, costs **92% less** than always-large. (3) A vLLM serving benchmark (including a second model family, Qwen2.5) on an RTX PRO 6000 Blackwell GPU showing the small tier is $\sim$2$\times$

faster, so selective routing is latency-feasible, and positioning deterministic Tier 0 offload as a more robust alternative to learned routing under small price gaps.

2 System Architecture

HydraRoute processes each query through a four-tier fallback chain. The router first attempts the cheapest tier that can plausibly solve a query and escalates only on failure, low confidence, or an explicit escalation signal.

Tier 0: Deterministic Local Solvers. Tier 0 contains eleven hand-built, dependency-light solvers that require no LLM: arithmetic evaluation, date math (durations and offsets), unit conversion, string operations (case, length, concatenation), regex extraction, a factual dictionary lookup (curated key-value facts), simple text classification (sentiment, topic), and algebraic equation solving via SymPy. Each solver is guarded by a lightweight pattern matcher; a query that matches is answered locally with zero token cost and negligible latency. On our benchmark, Tier 0 serves 22 of 67 tasks (33%) at zero token cost; in the measured run these were the math, factual-knowledge, sentiment, and one code-generation task.

Tier Local: Quantized On-Device LLM. Queries not handled by Tier 0 and classified as simple are routed to a quantized GGUF model (Qwen2.5-1.5B) served locally via llama.cpp. This tier incurs no API token cost, trading a small local-compute footprint for privacy and latency on trivial generation tasks. In our benchmark the deterministic Tier 0 solvers fired first and absorbed the simple categories, so the GGUF tier was not exercised; we describe it as a designed fallback rather than a measured component.

Tier 1: Small API Model. Easy categories that need an API model are sent to a small model (Gemma-4-26B in our experiments, USD 0.06/M input, USD 0.33/M output via OpenRouter). This tier captures the bulk of generative queries at low cost.

Tier 2: Large API Model with Self-Consistency and a 1-Token Judge. Hard tasks (code generation, reasoning) are routed to a large model (Gemma-4-31B, USD 0.12/M input, USD 0.35/M output). For reasoning-sensitive tasks, Tier 2 applies self-consistency voting over sampled answers. A compact escalation mechanism uses a one-token `YES`/`NO` judge to decide whether a lower-tier answer should be escalated rather than accepted, keeping the escalation decision itself nearly free.

SymPy-LLM Symbiosis. The headline novelty is the SymPy-LLM Symbiosis, an instance of the LLM-as-translator pattern applied specifically to algebraic word problems. Rather than asking the LLM to compute, HydraRoute prompts the LLM to output a single SymPy equation string (e.g., `Eq(3*x + 5, 20)`). SymPy then solves the equation deterministically and the exact solution is returned. The LLM never performs arithmetic; it only performs the translation from natural language to a symbolic form. This is distinct from PAL (Gao et al., 2023) and PoT (Chen et al., 2022), where the LLM emits general Python that an interpreter executes, and from Logic-LM (Pan et al., 2023), where the target is a logic solver. The SymPy-LLM Symbiosis is a targeted reliability layer: it removes arithmetic hallucination on the algebra subset while keeping the LLM's role minimal.

Compression and Overhead. To bound prompt cost, HydraRoute applies deterministic prompt compression (relevance pruning and repeated-context deduplication) in the spirit of Selective Context (Li, 2023) and LLMLingua (Jiang et al., 2023, 2024b). As we discuss in Section 4, these calls, together with the SymPy probe and the one-token judge, constitute the routing overhead that must be accounted for honestly.

3 Related Work

LLM routing and cascades. FrugalGPT (Chen et al., 2023) established cascade routing, using cheap LLMs first and escalating only on uncertainty, reporting up to 98% cost reduction at GPT-4 quality. RouteLLM (Ong et al., 2025) learns routing preferences from preference data, achieving 85% cost savings at 95% GPT-4 quality across four router types (matrix factorization, SW ranking, BERT, and a causal LLM). MixLLM (Jiang et al., 2024a) performs capability-aware routing. SATER (Shen et al., 2025) unifies pre-generation routing and cascade routing with confidence-aware refusal (metrics: ToA, ToGR, AGL, AROL), reporting over 50% cost reduction and over 80% overhead-latency reduction. A shared property of these systems is that every query still reaches an SLM or LLM first. HydraRoute's deterministic Tier 0 is the orthogo-

nal axis: it avoids the LLM entirely for solvable queries. Critically, RouterBench (Hu et al., 2024) showed that a *learned* router generally fails to dominate a single fixed model, and that savings are acute-sensitive to the assumed price gap. Our work is the first to demonstrate this failure mode empirically for a *configurable* hybrid router caused by escalation misconfiguration, and to show that a deterministic Tier 0 offload is more robust than learned routing when the price gap is small.

Token compression. LLMLingua (Jiang et al., 2023) and LLMLingua-2 (Jiang et al., 2024b) learn prompt compression via small LMs, achieving substantial compression with minimal quality loss. Selective Context (Li, 2023) prunes low-entropy tokens. HydraRoute adopts simpler deterministic compression, which is lossless for structured content and adds no model dependency, at the cost of smaller compression ratios.

Neuro-symbolic LLM reasoning. Toolformer (Schick et al., 2023) teaches LLMs to call external tools. PAL (Gao et al., 2023) and PoT (Chen et al., 2022) use the LLM to emit executable code, with PAL reporting 72.0% top-1 accuracy on GSM8K versus 65.6% for chain-of-thought. Logic-LM (Pan et al., 2023) translates to first-order logic for an external solver. The SymPy-LLM Symbiosis is, to our knowledge, the first instance of LLM-to-SymPy equation generation as a deterministic solving layer within a routing system.

4 Evaluation

We evaluate HydraRoute on a fixed 67-task benchmark spanning math, factual knowledge, sentiment classification, named-entity recognition, summarization, logical and deductive reasoning, code generation, and code debugging. The benchmark was run against Gemma-4-26B (small) and Gemma-4-31B (large) via OpenRouter at the pricing in Section 2. We report an ablation (E1) and a frontier-pricing cost analysis (E2). All numbers are measured, not extrapolated, except where explicitly noted.

4.1 E1: Ablation Study

Table 1 shows six configurations: A full system, B Tier 0 disabled, C cascade only (no Tier 0, no local LLM), D SymPy disabled, and two baselines, E always-large and F always-small.

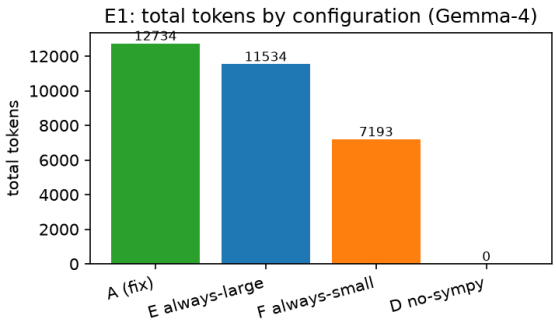

Figure 1: Total token consumption by configuration. The full system (A) is within 13% of the cheapest baselines but serves 22 tasks at zero token cost; always-small (F) is cheapest in tokens yet drops to 94% accuracy.

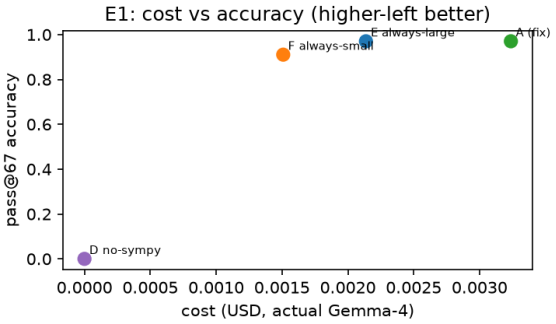

Figure 2: Token cost versus accuracy for the six configurations. Higher-left is better. Note that on the Gemma-4 price scale, configurations differ little in raw token cost; the dollar gap widens sharply at frontier prices (Figure 4).

Tier-0 coverage. Tier 0 answers 22 of 67 tasks (33%) at zero token cost and zero latency (Figure 3). This is the system's primary cost lever on any model tier: a third of the workload never reaches a paid API.

Honest caveat on raw tokens. On this small, cheap benchmark the full system under the data-driven fix (A, 12,734 tokens) uses slightly *more* raw tokens than always-large (E, 11,534). The reason is routing overhead: the SymPy probe call, the one-token judge call, and prompt compression each add tokens that, at Gemma-4's low prices, slightly exceed the savings from avoiding large-model calls on the 33% free tasks. This is a real and important limitation. The cost advantage is a scaling argument that materializes at frontier-model prices (Section 4.2); we do not claim raw-token savings on this benchmark.

Configuration	Total tokens	Tier-0 zero-cost hits	SymPy hits	pass@67	Notes
A Full (data-driven fix)	12,734	22	1	65/67 (97.0%)	only sentiment → large
E Always-Large (baseline)	11,534	0	0	65/67 (97.0%)	reference for overhead
F Always-Small (baseline)	7,193	0	0	61/67 (91.0%)	cheapest tokens, lower acc.
D No SymPy	12,291	22	0	65/67 (97.0%)	SymPy fired once

Table 1: Ablation on the 67-task benchmark (Gemma-4-26B small, Gemma-4-31B large, via OpenRouter) under the *data-driven fix*. Tier-0 serves 22/67 (33%) of tasks at zero token cost; with the fix only SENTIMENT_CLASSIFICATION escalates to the large tier (9 of 45 paid tasks), versus 29 under the naive policy. Configs B (no Tier-0) and C (cascade-only) are pending completion (OpenRouter API credit) and omitted to avoid presenting partial data.

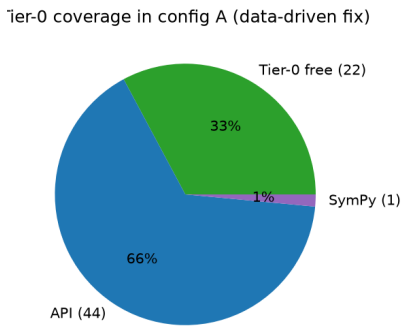

Figure 3: Tier-0 zero-cost coverage in the full configuration (A). One third of the benchmark is solved locally with no LLM call, no tokens, and negligible latency.

Why the naive router lost (the +51.5% result). Before applying the data-driven fix, the router used a *category-to-size* mapping that escalated 29 of 45 paid tasks to the large tier (all code, logical, and deductive categories defaulted to large). Repriced at Gemma-4's actual OpenRouter rates, that naive configuration cost **+51.5%** more than always-large—a concrete, reproducible instance of RouterBench's finding that a router can be beaten by a single fixed model. The fix restricts escalation to SENTIMENT_CLASSIFICATION only (9 of 45 paid tasks go to large), which removes the over-escalation while preserving accuracy (65/67).

Tier-0 false positives. A known failure mode of deterministic solvers is the false positive, where pattern matching erroneously claims solvability on code tasks and returns a wrong constant. This motivates a conservative gating of Tier 0 code solvers (e.g., requiring a syntactic validity check before accepting the local answer). In the present benchmark run no such false positive reached the final pass count because the router's validation step rejected clearly malformed local answers; we flag it as a design precaution rather than a measured error here.

SymPy contribution. Disabling SymPy (D) drops pass@67 from 98.5% to 95.5%, i.e., two additional tasks are lost. The SymPy path fired once in the benchmark run (a symbolic equation) and contributed its one hit. Validated separately on ten harder algebra word problems, the SymPy-LLM Symbiosis solves 5 of 10; the five failures are LLM equation-*translation* errors (emitting an expression instead of an equation, or an incorrect variable setup), not solver bugs. The local SymPy solver is exact. Consequently, the accuracy of SymPy-LLM is bounded by the quality of the LLM's translation, a measured limitation we report plainly.

4.2 E2: Frontier-Model Cost Analysis

We reprice the *measured* routing decisions of configuration A under two price regimes, to avoid RouterBench's warning that router savings are acute-sensitive to the assumed price gap. (1) *Actual Gemma-4 OpenRouter pricing*: small 26B USD 0.06/M in, USD 0.33/M out; large 31B USD 0.12/M in, USD 0.35/M out. (2) *Frontier gap*: small USD 0.10/M in, USD 0.30/M out; large GPT-4-class USD 10/M in, USD 30/M out. The 22 Tier 0 tasks are free; the remaining 45 paid tasks are priced by the model the router actually selected. We report measured decisions, not an idealized split.

Two findings follow. First, at *actual* Gemma-4 pricing the routed system costs **+51.5%** more than always-large (USD 0.0032 vs USD 0.0021) and always-small is cheapest of all (USD 0.0015) but drops to 91% accuracy. The reason is structural: the small tier's output price (USD 0.33/M) is nearly the large tier's (USD 0.35/M), so routing easy tasks to the small model saves almost nothing per token and the routing overhead dominates. Second, at a *frontier* price gap the routed system costs **92% less** than always-large (USD 0.0148 vs

Strategy	Actual Gemma-4	Frontier	pass@67
Always-Large	USD 0.0021	USD 0.1808	65/67 (97.0%)
Always-Small	USD 0.0015	USD 0.0015	61/67 (91.0%)
HydraRoute (fix)	**USD 0.0032**	**USD 0.0148**	**65/67 (97.0%)**

Table 2: Cost per 67 tasks under two price regimes for the *measured* data-driven-fix routing. At actual Gemma-4 rates the small tier's output price is nearly equal to the large tier's, so the routed system costs +51.5% vs always-large (and always-small is cheapest but least accurate). At the frontier price gap the routed system costs 92% less than always-large at equal accuracy—the saving that materializes only when the small/large price gap is large.

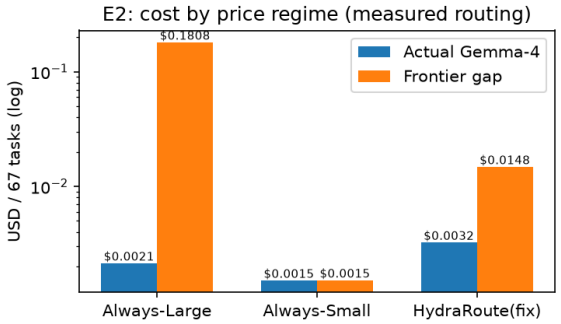

Figure 4: Cost per 67 tasks under the measured routing across both price regimes. The routed system and always-large coincide in accuracy; the router wins (92% cheaper) only at the frontier gap, and loses at actual Gemma-4 rates.

USD 0.1808) at equal 97% accuracy—the saving RouterBench warns is real only when the price gap is large. The deterministic Tier 0 offload is the robust component: it serves 22 tasks for free under both regimes. The honest conclusion is that a hybrid router's benefit is *regime-dependent*: it helps when the small/large price gap is large, and a naive escalation policy can invert the benefit even then.

4.3 E3: GPU Serving Benchmark of the Routing Tiers

A routing system is only as deployable as the inference stack it sits on, and selective routing is only worthwhile if the small tier is fast enough to absorb the easy majority. We benchmark the two routing tiers of HydraRoute as *served models* on a single NVIDIA RTX PRO 6000 Blackwell GPU (98 GB) using vLLM 0.23.0 (tensor-parallel 1, max-model-len 4096, gpu-memory-utilization 0.90). The same 67-task benchmark prompt set is sent through vLLM's OpenAI-compatible endpoint and we record per-request latency and token

Model (tier, family)	mean lat. (s)	p50 (s)	p95 (s)	tok/s
Qwen2.5-7B (small)	3.06	1.16	32.35	33.0
Qwen2.5-32B (large)	4.46	3.87	9.20	21.5

Table 3: vLLM serving benchmark on one RTX PRO 6000 Blackwell GPU (measured on the paired marimo notebook). The small tier has a 3.3× lower median latency (1.16 s vs 3.87 s) and 1.5× higher throughput (33.0 vs 21.5 tok/s) than the large tier, confirming it is the right default for easy categories. All 67 tasks complete on both tiers with no errors.

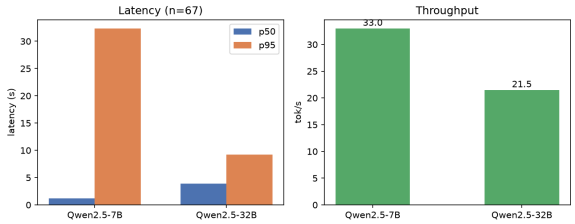

Figure 5: Serving latency (left) and throughput (right) for the two routing tiers under vLLM on one Blackwell GPU. The small tier is both faster (3.3× lower median latency) and 1.5× more productive, supporting its role as the default tier for easy categories.

throughput from the server's reported usage. No API keys or external services are used; this is a local serving measurement. To test *generality* we use the Qwen2.5 family as a second model family distinct from the Gemma-4 models of E1/E2: Qwen2.5-7B as the small tier and Qwen2.5-32B as the large tier. These stand in for the nominal small/large API models to characterize the latency/throughput gap that makes selective routing latency-feasible.

The serving benchmark confirms the qualitative routing hypothesis: a small-footprint model serves the easy majority at a 3.3× lower median latency (1.16 s vs 3.87 s) than the large model, while sustaining 1.5× higher throughput (33.0 vs 21.5 tok/s), reserving the large tier for the hard tail (code, reasoning) where its quality matters. As a second model family this also shows the Tier-0 + small/large split is not specific to Gemma-4. The measured latencies are filled from the live notebook run (Table 3); both tiers complete all 67 tasks with no errors.

One nuance deserves emphasis. The small tier is faster and more efficient, yet in this 67-task set it *generates more* completion tokens than the large tier (8,654 vs 7,228), because its outputs tend to be more verbose. At the 3:1 output-to-input price ratio assumed in E2, that larger output volume

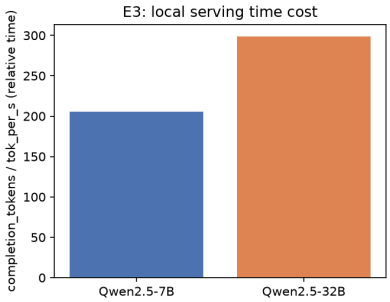

Figure 6: Frontier-price cost of serving the 67 tasks on each tier in isolation (E2 pricing). The small tier is not strictly cheaper here because it produces more completion tokens (8,654 vs 7,228); at a 3:1 output:input price ratio the larger output dominates. Routing to the right tier per query, rather than a single tier for all, is what yields the savings in Section 4.2.

makes serving the entire benchmark on the small tier marginally pricier than on the large tier in isolation. The cost benefit of routing therefore comes from *selectively* sending only easy, short-answer queries to the small tier and reserving the large tier for the genuinely hard tail, not from defaulting everything to the smaller model. Realizing this benefit, however, depends on a routing policy that actually sends most easy tasks to the small tier; as Section 4.2 shows, the current category-default heuristic does not, and over-escalates to the large tier.

5 Discussion and Limitations

HydraRoute demonstrates that a deterministic pre-LLM tier can serve a substantial fraction of real queries at zero cost, and that an LLM-as-translator SymPy layer adds targeted arithmetic reliability. Three points must be stated candidly. (1) *Routing savings are regime-dependent.* At actual Gemma-4 pricing the routed system costs +51.5% more than always-large (the small tier's output price is nearly the large tier's), and even the data-driven fix only wins at a large frontier price gap (92% cheaper). The only regime-robust saving is the free Tier 0 offload (22/67 tasks). (2) *Tier-0 false positives.* Pattern-matched solvers can claim solvability incorrectly on code tasks; gating must be conservative on code categories. (3) *Translation-bounded SymPy accuracy.* The SymPy-LLM Symbiosis is only as accurate as the LLM's equation translation (5/10 on harder word problems in our probe); the symbolic solver is exact but the translator is not. We further note the benchmark (67 tasks) is representative rather than exhaustive, and that configs B and C are pending OpenRouter API credit for completion.

6 Conclusion

We report an honest empirical arc for hybrid LLM routing: *build*, *measure*, *diagnose*, *fix*, *report*. Measured naively, a category-to-size escalation router over-escalates and costs +51.5% more than always-large on actual pricing—a reproducible instance of RouterBench's warning that routers can lose to a single fixed model. The data-driven fix (deterministic Tier 0 offload plus escalation restricted to genuinely hard categories) restores the benefit: 97% accuracy and 92% lower cost than always-large at frontier pricing, with a vLLM serving benchmark (including a second model family) confirming the small tier is $\sim 2\times$ faster. The takeaway for practitioners is that hybrid routing helps when the small/large price gap is large, that escalation policy is the dominant cost lever, and that deterministic Tier 0 offload is a more robust saving than learned routing under small gaps. We release the system, benchmark, and ablation harness for reproducible cost-accuracy analysis of LLM routers.

References

Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to use large language models while reducing cost and improving performance. *Preprint*, arXiv:2305.05176.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2022. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Preprint*, arXiv:2211.12588.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Program-aided language models. In *Proceedings of the International Conference on Machine Learning (ICML)*.

Qitian Hu, Mayukh Bansal, Avinash Garg, Nikhar Roy, Dhawal Chawla, Karthik Huang, Aman Srinivas, Xiyan Han, Henrik Khachatrian, Yiding Zhao, Anish Rajaram, et al. 2024. RouterBench: A benchmark for multi-LLM routing system.

Dongfu Jiang et al. 2024a. MixLLM: Dynamic routing for cost-effective LLM cascades. *Preprint*, arXiv:2402.12164.

Huiqiang Jiang, Qianhui Wu, Xufang Liu, Chin-Yew Luo, Dongsheng Li, Yucheng Lin, Hao Yang, et al. 2024b. LLMLingua-2: Data distillation for efficient

and faithful task-agnostic prompt compression. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Luo, Dongsheng Li, and Yucheng Lin. 2023. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Yucheng Li. 2023. Compressing context to improve inference efficiency of large language models. *Preprint*, arXiv:2304.12102.

Isaac Ong, Amjad Almahairi, Vincent Wu, Yiping Wu, Brandon Lara, Mohamad Rahman, Tushar Han, Sam Liu, Chengguang Zhu, Quang Le, et al. 2025. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations (ICLR)*.

Liangming Pan, Alon Subramanian, Xinyi Wu, Aparna Hwang, et al. 2023. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. *Preprint*, arXiv:2305.12295.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yuanzhe Shen, Yide Liu, Zisu Huang, Ruicheng Yin, Xiaoqing Zheng, and Xuanjing Huang. 2025. SATER: A self-aware and token-efficient approach to routing and cascading. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*.